

SENAC-RS

CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

FRAMEWORKS WEB

PEDIDO DE DIÁRIAS EM ÓRGÃOS PÚBLICOS

Desenvolvimento de uma API para automação de processos administrativos

Autor: Adriano Nunes dos Santos

Professor: Luis Henrique Ries

Canoas, Rio Grande do Sul
Maio de 2026

1. Introdução

Este relatório descreve o desenvolvimento de uma aplicação backend voltada para a gestão e o cálculo de diárias no setor público. Utilizando o framework **NestJS**, o projeto aplica princípios de arquitetura em camadas para garantir que a lógica de negócio seja independente e escalável. A escolha do tema justifica-se pela complexidade intrínseca das regras de indenização por deslocamento de servidores, que dependem diretamente de legislação variável.

2. Definição do Problema

O processo de solicitação de diárias em prefeituras, câmaras, tribunais de justiça, ministérios públicos e tais como, é normalmente estigmatizado por falta de padronização. Atualmente, muitos desses processos são realizados manualmente ou por ferramentas (planilhas) sem validação automática ou qualquer tipo de auditoria, o que acarreta em:

- **Inconsistência nos valores:** erros manuais ao aplicar acréscimos/decréscimos conforme a localidade ou horário.
- **Lentidão no fluxo:** a necessidade de conferência humana para cada cálculo atrasa o ressarcimento do servidor.
- **Dificuldade de auditoria:** sistemas sem regras de negócio rígidas dificultam o controle de gastos públicos.

3. Solução Proposta

A solução desenvolvida é uma API (application Programming Interface) que centraliza a inteligência do cálculo de diárias. O sistema permite que a administração cadastre os pedidos e receba, em tempo real, o valor exato a ser pago, já processado conforme as regras vigentes.

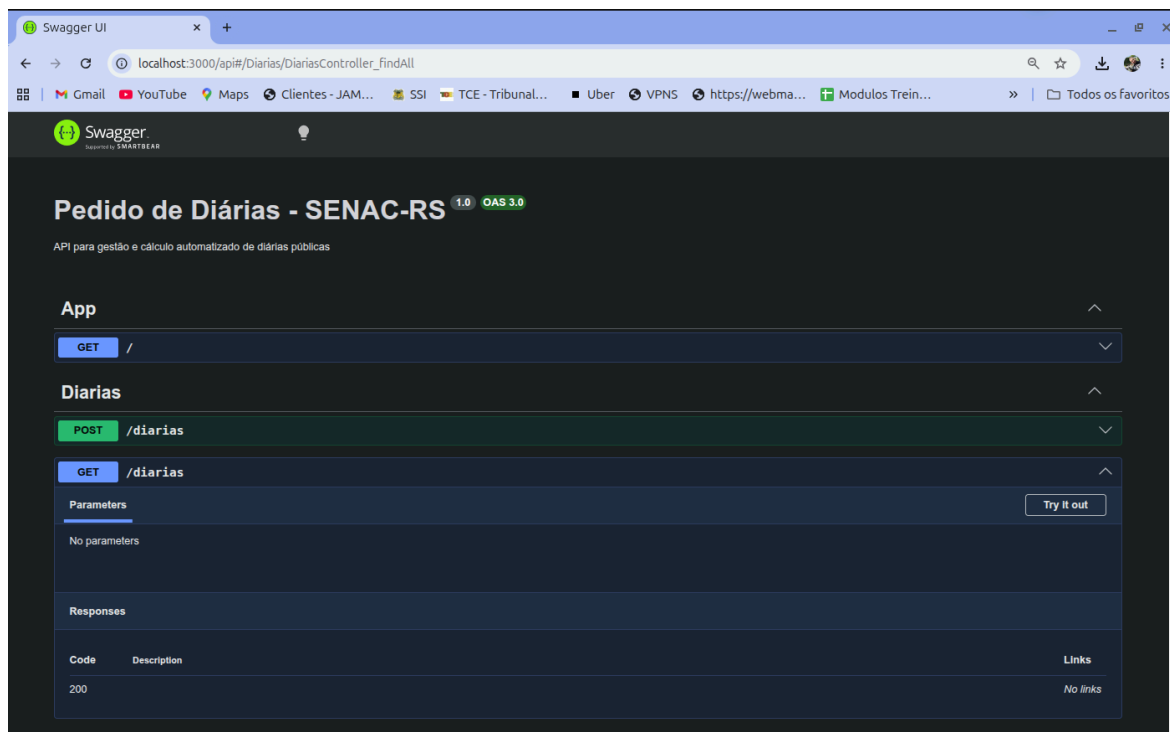
3.1 Requisitos Funcionais

RF01 - Cálculo Automático: o sistema deve calcular o valor total com base no cargo do servidor.

RF02 - Adicional de Destino: deve ser aplicado um acréscimo de 30% para destinos como Porto Alegre e Brasília.

RF03 - Fator Pernoite: o sistema deve reduzir o valor em 50% caso a viagem não possua pernoite (bate-volta)

RF04 - Validação de Dados: o sistema deve impedir o processamento de cargos que não estejam previstos na legislação cadastrada.



4. Arquitetura do Sistema

A aplicação foi estruturada seguindo o padrão de **Arquiteturas em Camadas (Layered Architecture)**, o que facilita a manutenção e a testabilidade do código.

4.1 Divisão de Camadas

Camada de Controladores (Controllers): responsável por receber as requisições externas e validar o formato dos dados.

Camada de Serviços (Services/Domain): onde reside o 'core' da aplicação. Toda a lógica de cálculo de porcentagem e validação de cargos foi isolada nesta camada, mantendo-a pura e independente.

Camada de Entidades (Entities): define os modelos de dados (Servidor, Destino, Valor Total) que transitam pela aplicação.

4.2 Decisões Técnicas

A escolha pelo **NestJS** foi motivada pela sua arquitetura 'opinativa', que impõe uma organização modular natural. O uso de **TypeScript** trouxe segurança através da tipagem estática, reduzindo erros em tempo de execução, especialmente críticos em sistemas financeiros e administrativos (e transparência).

5. Modelagem de Domínio e Regras de Negócio

Para este projeto, as regras de negócio foram parametrizadas da seguinte forma:

- **Tabela de Valores por Cargo:**
 - Operacional: R\$ 200,00
 - Técnico: R\$ 350,00
 - Gestão: R\$ 600,00
- **Lógica de Acréscimo:** utilização de uma lista de destinos especiais para a aplicação de multiplicador de 1.3 (30%).
- **Lógica de Redução:** aplicação de fator 0.5 (50%) para pedidos com temPernoite = false.

```
src > diarias > ts diarias.service.ts > DiariasService > findAll
1  import { Injectable, BadRequestException } from '@nestjs/common';
2  import { Diaria } from './entities/diaria.entity';
3
4  @Injectable()
5  export class DiariasService {
6    // lista na memória para guardar os pedidos (sem bd ainda)
7    private diarias: Diaria[] = [];
8
9    create(dadosDoPedido: any) {
10     // Definição de Valores e Validação de Cargo | Criamos um mapa de cargos para facilitar a consulta e a validação
11     const cargosValidos = {
12       'OPERACIONAL': 200,
13       'TECNICO': 350,
14       'GESTAO': 600
15     };
16
17     // Segurança: Se o cargo não existir o sistema da um break aqui
18     if (!cargosValidos[dadosDoPedido.cargo]) {
19       throw new BadRequestException(
20         `O cargo '${dadosDoPedido.cargo}' não é válido. Escolha entre: OPERACIONAL, TECNICO ou GESTAO.`
21       );
22     }
23
24     // Se passou pela trava, pega o valor correto
25     const valorBase = cargosValidos[dadosDoPedido.cargo];
26
27     // Regra do Destino: Se for Porto Alegre ou Brasília aumenta 30%
28     let valorCalculado = valorBase;
29
30     // conversão para upper na entrada, para não deixar margem para o usuário
31     const destinoInformado = dadosDoPedido.destino.toUpperCase();
32     const destinosEspeciais = ['PORTO ALEGRE', 'BRASILIA'];
```

continua na imagem abaixo: diarias.service.ts

```

33
34     if (destinosEspeciais.includes(destinoInformado)) {
35         valorCalculado = valorCalculado * 1.30; // Aplica acréscimo de 30%
36     }
37
38     // Pernoite: Se for bate-volta paga só metade (50%)
39     if (dadosDoPedido.temPernoite === false) {
40         valorCalculado = valorCalculado * 0.5;
41     }
42
43     // Criação do objeto final com o ID e o valor calculado
44     const novaDiaria: Diaria = {
45         id: this.diarias.length + 1,
46         ...dadosDoPedido,
47         valorTotal: valorCalculado, // Produto da lógica de negócio
48     };
49
50     // salvando em memória a lista
51     this.diarias.push(novaDiaria);
52
53     return novaDiaria;
54 }
55
56 // Função que lista as diárias cadastradas
57 findAll() {
58     return this.diarias;
59 }
60

```

continuação da imagem acima: diarias.service.ts

6. Tratamento de Exceções

A aplicação implementa o tratamento de erros globais, isto é, caso uma requisição envie dados inválidos (ex: um cargo inexistente), o sistema utiliza o `BadRequestException` para retornar uma resposta clara e estruturada ao usuário, evitando falhas inesperadas no servidor.

7. Ferramentas Utilizadas

- **Framework NestJS:** para a estrutura robusta do backend.
- **Swagger (OpenAPI):** implementado para a documentação interativa da API.
- **GitHub:** Utilizado para versionamento e controle de código.

8. Uso Crítico de Ferramentas de IA

A Inteligência Artificial foi utilizada de forma estratégica durante o desenvolvimento. O uso ficou restrito em tirar dúvidas e:

- **Estruturação Arquitetural:** auxílio na definição de módulos e injeção de dependência do NestJS.
- **Aceleração de Código:** geração de boilerplate (base/andaime) para as entidades e controladores.
- **Análise Crítica:** a decisão final sobre as regras de negócio e a supervisão da lógica de cálculo foram mantidas sob controle humano, garantindo que o código gerado atendesse especificamente ao domínio público solicitado (com exemplo em experiência e dados maquiados na empresa Thema Informática e seu cliente, Tribunal de Justiça do Estado de Santa Catarina).

9. Considerações Finais

O desenvolvimento do projeto **Pedido de Diárias em Órgãos Públicos** permitiu consolidar os conhecimentos de Frameworks Web e a arquitetura de software. A implementação de uma API escalável demonstra que é possível modernizar processos burocráticos através de tecnologias modernas.

A principal contribuição deste trabalho é a prova de conceito de que a automação de regras de negócio complexas pode reduzir drasticamente a margem de erro em pagamentos públicos. Como trabalho futuro, vislumbra-se a integração com banco de dados relacionais e a implementação de autenticação JWT para maior segurança.